Artifact Abstract: How Effective Is Coverage-Guided Fuzzing to Test Deep Learning Library APIs?

Feiran (Alex) Qin[1], M M Abid Naziri[1], Hengyu Ai[2], Saikat Dutta[3], Marcelo d'Amorim[1]
[1]North Carolina State University [2]ShanghaiTech University [3]Cornell University

Paper Title

How Effective Is Coverage-Guided Fuzzing to Test Deep Learning Library APIs?

Artifact Purpose

This artifact accompanies the above paper, accepted to the ICST 2026 Research Track. It contains the complete implementation of FlashFuzz, a framework that employs coverage-guided fuzzing (CGF) to test Deep Learning (DL) library APIs at scale. FlashFuzz leverages Large Language Models (LLMs) to automatically synthesize API-level test harnesses for PyTorch and TensorFlow, enabling effective CGF for DL libraries.

The artifact includes:

- Source code for FlashFuzz (experiment runner, Docker orchestration, test harness generation).
- Pre-generated test harnesses for 1,271 PyTorch and 786 TensorFlow APIs.
- Dockerfiles that build PyTorch and TensorFlow from source with coverage instrumentation and fuzzer support.
- Baseline configurations for ACETest, PathFinder, and TitanFuzz.
- Ablation study harness variants (with/without helper functions, with/without documentation).
- Jupyter notebooks and post-processing scripts to reproduce all figures and tables in the paper.

Badges Applied For

Artifact Available. The artifact is publicly archived on Zenodo (DOI: 10.5281/zenodo.18884206) under the MIT license, ensuring long-term public availability. The development repository is at https://github.com/ncsu-swat/FlashFuzz/tree/artifact-evaluation.

Artifact Reviewed. We apply for the Reviewed badge on all four criteria:

- **Documented:** The repository includes a detailed `README.md` with step-by-step instructions for setup, a kick-the-tires quick test (30 min), and full experiment reproduction.
- **Consistent:** The artifact directly produces the results reported in the paper (coverage comparisons, ablation study, validity analysis, and bug detection).
- **Complete:** All components needed to reproduce the paper's experiments are included: source code, test harnesses, Docker images, baseline configurations, API lists, and plotting scripts.
- **Exercisable:** Pre-built Docker images are available on GitHub Container Registry. The kick-the-tires evaluation can be completed in 10 minutes with pre-built images. All scripts and notebooks can be executed to reproduce the paper's results.

Technology Skills

Reviewers should be familiar with:

- **Docker** — building and running containers.
- **Python** — running Python scripts from the command line.
- **Jupyter Notebooks** — executing notebook cells to reproduce plots.
- **Linux command line** — basic shell operations.

No knowledge of fuzzing internals, C++ test harness code, or DL library internals is required to run the artifact.

Access and Environment

Zenodo archive: DOI: 10.5281/zenodo.18884206

Repository: https://github.com/ncsu-swat/FlashFuzz/tree/artifact-evaluation

Pre-built Docker images: Available on GitHub Container Registry under `ghcr.io/ncsu-swat/flashfuzz:*`. Pulling images avoids the multi-hour build step.

Hardware requirements:

Resource	Minimum	Recommended
CPU cores	8	50+ (for parallel fuzzing)
RAM	16 GB	64 GB+
Disk	50 GB free	200 GB+
GPU	Not required	Optional (NVIDIA + Container Toolkit)

Operating system: Linux (tested on Ubuntu 22.04). Docker must be installed and accessible without `sudo`.

Software dependencies: Python 3.10+, `tqdm`, `bs4`, `regex`. A `pixi.toml` is provided for one-command environment setup via Pixi.

Setup time: 10 minutes with pre-built Docker images (pull + pixi install). 30 minutes if building the kick-the-tires image from source. Several hours if building all images from source.